Lec 70
Samarth Sir Video

27 Dec, 2024

Pehle project set karo vite se :—

Task 1 :- create karo todo component (title, desc) + input box

```
import React from 'react'

function Todos() {
  return (

    <div>
        <SingleTodo titl="go to place-1" desc="place-1 is amazing" />
        <SingleTodo titl="go to place-2" desc="place-2 is amazing" />
        <SingleTodo titl="go to place-3" desc="place-3 is amazing" />
    </div>

  )
}

function SingleTodo({titl, desc}){

    return (
        <div>
            <h1>{titl}</h1>
            <h1>{desc}</h1>
        </div>

    )
}

export default Todos
```

L1 ←

→ pehle hamne ye simple sa code likha. Ab hum isse optimise karenge.

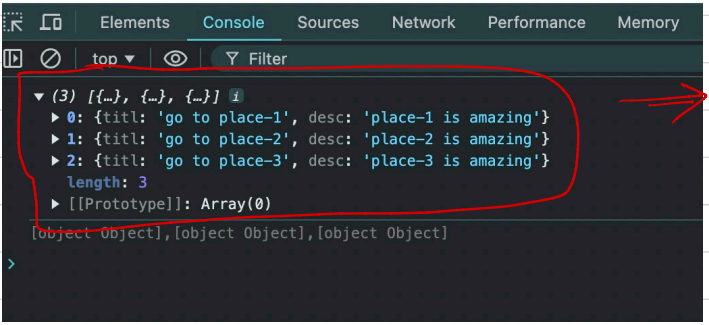
Hame L1 par repeated calling avoid karni hai.

Hum title, desc ko array mei use karenge.
Array of objects. And wo array Single Todo mei pass hoga.

Ab kyuki Array ki value increase ho sakti hai, isliye we will use State().

↳ new task/input/todo karne par.

```jsx
import React, { useState } from 'react'

function Todos() {

    let arr = [
        { titl: "go to place-1", desc: "place-1 is amazing" },
        { titl: "go to place-2", desc: "place-2 is amazing" },
        { titl: "go to place-3", desc: "place-3 is amazing" },
      ];

    let [arrays, setArrays] = useState(arr);

    return (

    <div>
        <SingleTodo todo= {arrays}/>
    </div>

  )
}

function SingleTodo({todo}){
    console.log(todo);

}

export default Todos
```



todo is my array.

Task 2 :- 3 functions calling ki jagah → iterate karo array par and then call function only one time.

Ab kyuki todo array hai isliye map() use karna padega to display to do items.

Note:- you cannot display objects in Browser DOM when using React; isliye hamne map() use kiya.

```jsx
import React, { Fragment, useState } from 'react'

function Todos() {

    let arr = [
        { titl: "go to place-1", desc: "place-1 is amazing" },
        { titl: "go to place-2", desc: "place-2 is amazing" },
        { titl: "go to place-3", desc: "place-3 is amazing" },
      ];

    let [arrays, setArrays] = useState(arr);

    return (

    <div>
        <SingleTodo todo= {arrays}/>
    </div>

  )
}

function SingleTodo({todo}){
    // console.log(todo);

    return(

        <Fragment>

        {todo.map((item, index) => {
          return (
            <div>
              <h1> {item.titl} </h1>
              <h1> {item.desc} </h1>
            </div>
          );
        })}

      </Fragment>

    )

}

export default Todos
```

→ Todos.jsx

localhost:5173

go to place-1

place-1 is amazing

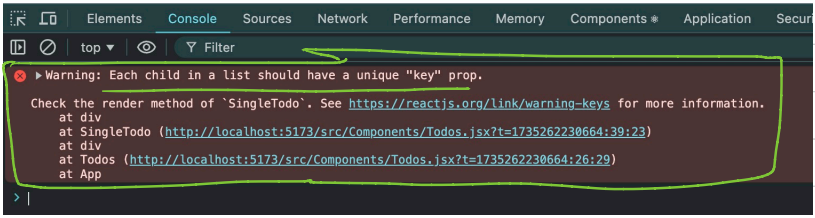
go to place-2

place-2 is amazing

go to place-3

place-3 is amazing

⇒ o/p



↳ why to add key ?

↓ Answer

(26:00)

In React, the `key` prop is essential when rendering lists of elements to help React identify which items have changed, been added, or removed. This is crucial for efficiently updating the UI.

Why is `key` needed?

When React renders a list of elements, it needs a way to distinguish between them. Without unique keys, React may:

1. **Re-render unnecessarily**: React might recreate all the list elements even if only one changes, which is inefficient.
2. **Cause UI bugs**: If React cannot reliably identify an element, it might mix up their state or order.

How does `key` help?

- A unique `key` allows React to track each element in the list.
- If an item in the list changes, React upd↓ only that specific item instead of re-rendering the entire list.

```
<Fragment>

      {todo.map((item, index) => {
        return (
          <div key= {index}>
            <h1> {item.titl} </h1>
            <h1> {item.desc} </h1>
          </div>
        );
      })}

    </Fragment>
```

→ for each element, key should be unique & different.

Isliye use index here. index will keep changing for each element.

```
import React, { Fragment, useState } from 'react'

function Todos() {

    let arr = [
        { titl: "go to place-1", desc: "place-1 is amazing" },
        { titl: "go to place-2", desc: "place-2 is amazing" },
        { titl: "go to place-3", desc: "place-3 is amazing" },
      ];

    let [arrays, setArrays] = useState(arr);

    return (

    <div>
        <SingleTodo todo= {arrays}/>
    </div>

  )
}

function SingleTodo({todo}){
    // console.log(todo);

    return(

        <Fragment>

        {todo.map((item, index) => {
          return (
            <div key= {index}>
              <h1> {item.titl} </h1>
              <h1> {item.desc} </h1>
            </div>
          );
        })}

      </Fragment>

    )

}

export default Todos
```

→ Todos.jsx

Task 3 :- input + button (to add todo) → new todo (create and array push.

```jsx
import React, { Fragment, useState } from 'react';

function Addd() {
    let arr = [
        { titl: "go to place-1", desc: "place-1 is amazing" },
        { titl: "go to place-2", desc: "place-2 is amazing" },
        { titl: "go to place-3", desc: "place-3 is amazing" },
    ];

    let [array, setArr] = useState(arr);

    return (
        <div>
            <AddTodo arrays={array} sArr={setArr} />
            <Todo todo={array} />
        </div>
    );
}

// Component to add a new todo
function AddTodo({ arrays, sArr }) {
    let [inp1, setInp1] = useState('');
    let [inp2, setInp2] = useState('');

    function changeInp1(e) {
        setInp1(e.target.value);
    }

    function changeInp2(e) {
        setInp2(e.target.value);
    }

    function handleAddTodo() {
        sArr([...arrays, { titl: inp1, desc: inp2 }]);
        setInp1(''); // Clear the input fields after adding
        setInp2('');
    }

    return (
        <div>
            <input onChange={changeInp1} type="text" value={inp1} />
            <input onChange={changeInp2} type="text" value={inp2} />
            <button onClick={handleAddTodo}>Add Todo</button>
        </div>
    );
}

// Component to display all todos
function Todo({ todo }) {
    return (
        <div>
            {todo.map((item, index) => (
                <div key={index}>
                    <h1>{item.titl}</h1>
                    <h1>{item.desc}</h1>
                </div>
            ))}
        </div>
    );
}

export default Addd;
```

→ i1

→ i2

→ Addd.jsx

l1 and l2 are uncontrolled inputs without onChange = {changeInp1}.

l1 & l2 inputs ke changes ko track karne ke liye / to make them controlled input, we use onChange.

1. Initial Setup

- **Component Hierarchy:**
  - The main component is `Addd`, which renders two child components:
    1. `AddTodo`: Handles adding new todos to the list.
    2. `Todo`: Displays the current list of todos.
- **Initial Data:**

```javascript
let arr = [
    { titl: "go to place-1", desc: "place-1 is amazing" },
    { titl: "go to place-2", desc: "place-2 is amazing" },
    { titl: "go to place-3", desc: "place-3 is amazing" },
];
```

  - A hardcoded array of objects representing the initial list of todos.
- **State Management:**

```javascript
let [array, setArr] = useState(arr);
```

  - `array`: A state variable to store the current list of todos, initialized with `arr`.
  - `setArr`: A function to update the `array` state.

2. Execution Flow

Step 1: Render `Addd`

- When the `Addd` component is rendered:
  - The initial `array` state contains the three hardcoded todos from `arr`.
  - The `AddTodo` and `Todo` components are rendered, with `array` passed as a prop to both.

Step 2: `AddTodo` Component

- **Purpose:** Provides two input fields and a button to add a new todo.
- **State:**
  - `inp1`: Stores the value of the title input field.
  - `inp2`: Stores the value of the description input field.

- **User Interaction:**

1. **Typing in Input Fields:**

   - When the user types in the first input field:

```javascript
<input onChange={changeInp1} type="text" value={inp1} />
```

     - `changeInp1` is called:

```javascript
function changeInp1(e) {
    setInp1(e.target.value);
}
```

       - Updates `inp1` with the current value of the input.

   - Similarly, for the second input field:

```javascript
<input onChange={changeInp2} type="text" value={inp2} />
```

     - Updates `inp2`.

2. **Adding a Todo:**

   - When the "Add Todo" button is clicked:

```javascript
<button onClick={handleAddTodo}>Add Todo</button>
```

     - `handleAddTodo` is executed:

```javascript
function handleAddTodo() {
    sArr([...arrays, { titl: inp1, desc: inp2 }]);
    setInp1('');
    setInp2('');
}
```

       - Creates a new todo object with `inp1` and `inp2` as the title and description.
       - Updates the `array` state (`sArr`) by appending the new todo to the existing list.
       - Resets the input fields to empty.

Step 3: `Todo` Component

- **Purpose:** Displays the current list of todos.
- **Props:**
  - `todo`: The current list of todos, passed from the `Addd` component (`array` state).
- **Rendering:**

javascript Copy code

```
{todo.map((item, index) => (
    <div key={index}>
        <h1>{item.titl}</h1>
        <h1>{item.desc}</h1>
    </div>
))}
```

  - Loops through the `todo` array using `map`.
  - For each `item`, renders a `div` with its title (`item.titl`) and description (`item.desc`).
  - Uses the `index` as the `key` for each `div`.

Wrapper Component :- → It is a parent component jo child component ko wrap karta hai.

→ We dont use it much. Ise generally design ke context mei use karte hai; taaki sabhi child ko ek saath wrap karke design de do wrapper component ko.

We can send the entire child component as a prop in wrapper component.

```
import React from 'react'                                    → l 0

function WrapperComponent({compo}) {

  return (
    <div style={{border: "2px solid red", backgroundColor:
"yellow"}} >                                                 → l 1
        {compo}                                              → l 5
    </div>
  )
}

export function ChildComponent(){                            → l 2

    return(
        <div>
            <h1>CHILD HU MAI 1</h1>
        </div>
    )
}

export default WrapperComponent                              → l 3
```

→ Wrapper.jsx

```
import React from 'react'
// import Todos from './Components/Todos'
// import Addd from './Components/Addd'
import WrapperComponent, {ChildComponent} from "./
Components/Wrapper";

function App() {
  return (
    // <Todos/>                                              → l 4
    // <Addd/>
    <WrapperComponent compo={<ChildComponent/>} />
  )
}

export default App
```

→ App.jsx

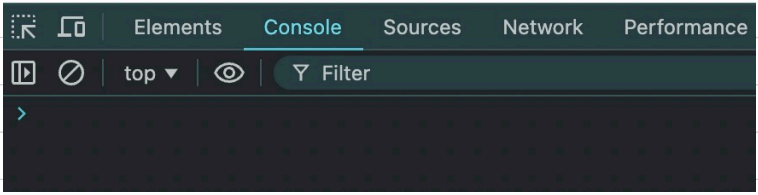
→ l4 par hamne Wrapper Component function ko call kiya with parameter compo jisme hamne Child Component ko wrap kar diya.

→ l1 par we can see double braces {{ }} . it is a part of syntax → { } → matlab evaluation.

→ l5 par hamne Child Component ko call kiya.

→ l2 par hamne Child Component ko export kiya.

↳ child component got wrapped
Note that design was applied on wrapper component.



Let's wrap another child:-

```jsx
import React from 'react'

function WrapperComponent({compo, hello}) {

  return (
    <div style={{border: "2px solid red", backgroundColor: "yellow"}} >
        {compo}
        {hello}
    </div>
  )
}

export function ChildComponent(){

    return(
        <div>
            <h1>CHILD HU MAI 1</h1>
        </div>
    )
}

export function ChildComponent2(){

    return(
        <div>
            <h1>CHILD HU MAI 2</h1>
        </div>
    )
}

export default WrapperComponent
```

Wrapper.jsx

```jsx
import React from 'react'
// import Todos from './Components/Todos'
// import Addd from './Components/Addd'
import WrapperComponent, {ChildComponent, ChildComponent2} from "./Components/Wrapper";

function App() {
  return (
    // <Todos/>
    // <Addd/>
    <WrapperComponent compo={<ChildComponent/>} hello = {<ChildComponent2/>}/>
  )
}

export default App
```

App.jsx

We dont use Wrapper thing much in real life.

Q Mai chahta hu Actual Wrapper mei Chota Wrapper aa jaye and chhote wrapper mei kuch content aa jaye jo independent hai (not part of any component)

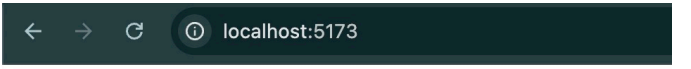
Sol:-

```jsx
import React from "react";

function ActualWrapper({ children }) {
  return (
    <div style={{ border: "2px solid red", backgroundColor:
"yellow" }}>
      {children}
    </div>
  );
}

export function ChotaWrapper({ children }) {
  return (
    <div>
      <h1>hello h1</h1>
      <h2>hello h2</h2>
      {children}
    </div>
  );
}

export default ActualWrapper;
```

→ ActualWrapper.jsx

```jsx
import React from 'react'
// import Todos from './Components/Todos'
// import Addd from './Components/Addd'
import WrapperComponent, {ChildComponent, ChildComponent2} from "./Components/Wrapper";
import ActualWrapper, { ChotaWrapper } from "./Components/ActualWrapper";

function App() {
  return (

    <React.Fragment>
      {/* <Todos /> */}
      {/* <Addd /> */}
      {/* <WrapperComponent compo={<ChildComponent/>} hello = {<ChildComponent2/>}/> */}
      {/* <WrapperComponent compo={<ChildComponent />} /> */}
      {/* <WrapperComponent compo={<ChildComponent2 />} /> */}

      {/* better approach ✅ */}
      <ActualWrapper>
        <ChotaWrapper>
          <h1>hello from h1</h1>
          <h2>hello from h1</h2>
        </ChotaWrapper>
      </ActualWrapper>

    </React.Fragment>
  )
}

export default App
```

→ App.jsx

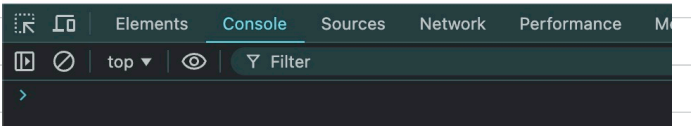


hello h1

hello h2

hello from h1

hello from h1



Final DOM Structure

After all components have rendered, the DOM will look like this:

html Copy code

```
<div style="border: 2px solid red; background-color: yellow;">
  <div>
    <h1>hello h1</h1> <!-- From ChotaWrapper -->
    <h2>hello h2</h2> <!-- From ChotaWrapper -->
    <h1>hello from h1</h1> <!-- From App's children to ChotaWrapper -->
    <h2>hello from h1</h2> <!-- From App's children to ChotaWrapper -->
  </div>
</div>
```

(1:42:30) Ab talk memo(), useState() hogya

useEffect() :- It is a hook which works on side effect.

useEffect is a React Hook that allows you to perform side effects in function components. Side effects include things like fetching data, updating the DOM, subscribing to events, and setting up timers.

useEffect replaces lifecycle methods like componentDidMount, componentDidUpdate, and componentWillUnmount in class components.

useEffect is a React Hook that allows you to perform side effects in function components. Side effects include things like fetching data, updating the DOM, subscribing to events, and setting up timers.

pls simplify it

`useEffect` is a tool in React that lets you do extra things in your app after it shows something on the screen. These "extra things" can include:

- Getting data from the internet (fetching data).
- Changing something on the page (updating the DOM).
- Setting up things like timers or event listeners (e.g., listening for a button click or window resize).

In simple terms, `useEffect` lets you add logic that should happen *after* React has updated the screen.

useEffect() accepts cb function and an array.

useEffect(() => {}, [])

iss array ko dependency array kahenge:

→ useEffect()

Q when does this run?

tab

useEffect() first time ∧ chalta hai jab aapka component render hota hai.

ye dusri baar tab chalega jab mai dependency array ke ander koi state ya prop likhta hu and wo state / prop change hote hai.

github profile fetching profile search karo google pe.

jab bhi fetch, setTimeout, setInterval use karna ho, use karo useEffect().

(2:00:00)

```
import React from 'react'
import Github from './Components/Github'

function App() {
  return (

    <div>
      <Github username="dhruvalgupta2003" />
      <Github username="mohitpanchal09" />
    </div>

  )
}

export default App
```

→ App.jsx

```jsx
import React, { useState, useEffect } from 'react'

function Github({ username }) {

    let [user, setUser] = useState({imgUrl: "", followers: 0, following: 0});

    useEffect( ()=>{
        fetch(`https://api.github.com/users/${username}`). then(async
function(res){
            let data = await res.json();
            console.log(data);
        })
    }, []);

    return (
        <div>
            <img src={user.imgUrl} alt="img" />
            <figure>
                <p>Username: {}</p>
                <p>Followers: {user.followers}</p>
                <p>Followings: {user.following}</p>
            </figure>
        </div>
    )
}

export default Github
```

l1 → l2 → l3

↳ github.jsx

Iss program mei hum github api use karke github users ki information web par display karana chahte hai.

l1 par hamne {username} destructure kiya

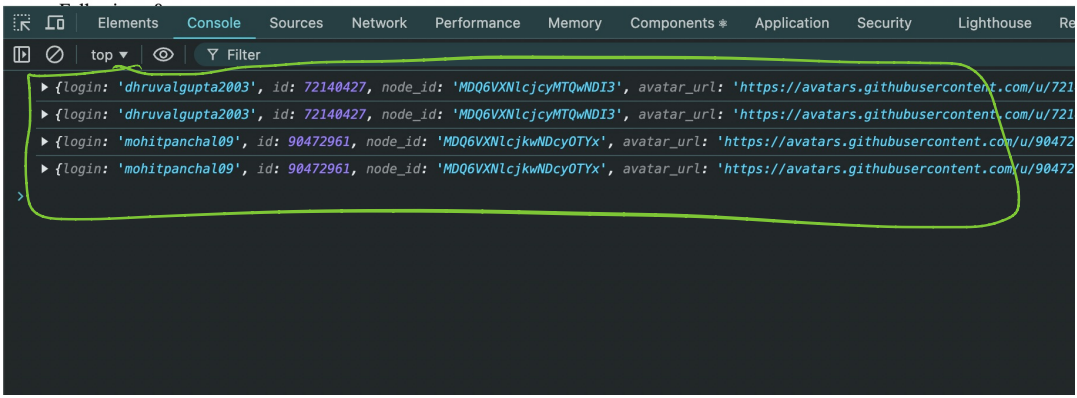
l2 → yaha "user" is an object containing following:-

imgUrl : ""
followers: 0
following : 0

l3 → kyuki mujhe Api calling karni hai, isliye me

will use useEffect(). fetch promise return karega isliye ; .then () use karna padega.

Then hamne us data ko console kara diya



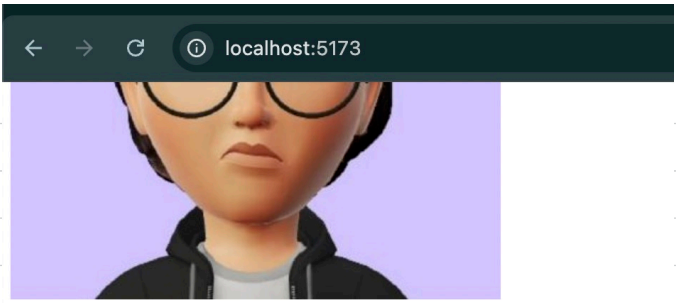
Final code :-

```
import React, { useState, useEffect } from 'react'

function Github({ username }) {

    let [user, setUser] = useState({imgUrl: "", followers: 0, following: 0});

    useEffect( ()=>{
        fetch(`https://api.github.com/users/${username}`). then(async function(res){
            let data = await res.json();
            let {avatar_url, followers, following} = data;
            setUser(() => {
                return {
                  imgUrl: avatar_url,
                  followers: followers,
                  following: following,
                };
              });
        })
    }, []);

    return (
        <div>
            <img src={user.imgUrl} alt="img" />
            <figure>
                <p>Username: {username}</p>
                <p>Followers: {user.followers}</p>
                <p>Followings: {user.following}</p>
            </figure>
        </div>
    )
}

export default Github
```

l1

l2

l3

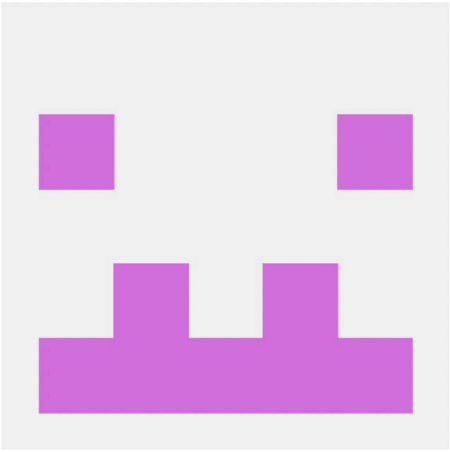


⇒ o/p

Username: dhruvalgupta2003

Followers: 9

Followings: 24



Username: mohitpanchal09

Followers: 3

Followings: 5

l 1 par hamne jo bhi destructure kiya, use l 2 par maine setUser function ki madad se user object mei daal diya.

l 2 ⇒ aap directly object ki value set nahi, sakte, you will need to use ek function. Hamne object return kara diya using function inside setUser fnc.

Note:- l 3 → par hamne empty array diya hai.

Dependency array mei hum generally state and prop likhte hai.

agar aapne useEffect ke cb function mei state ya prop use kiya hai; sirf tabhi aap use dependency array mei daaloge; others-wise nahi.

Hamne apne useEffect() ke code mei "users" state variable ka istemal nahi kiya, isliye use dependency array mei nahi dala.

Agar mai [user] dependency array mei daal deta, then jab jab "user" ke ander change hota, tab tab useEffect firse chalta → matlab firse fetching hoti.

```
let [user, setUser] = useState({imgUrl: "", followers: 0, following: 0});

  useEffect( ()=>{
      fetch(`https://api.github.com/users/${username}`). then(async function(res){
          let data = await res.json();
          let {avatar_url, followers, following} = data;
          // console.log(user);
          setUser(() => {
              return {
                imgUrl: avatar_url,
                followers: followers,
                following: following,
              };
            });
      })
  }, [user]);
```

l1 → l2

agar hamne useEffect() ke ander user use kiya hota (jaise l1 par), tab l2 par hame [user] likhna padta.